NHẬP MÔN HỌC MÁY VÀ KHAI PHÁ DỮ LIỆU

TÀI LIỆU ĐỌC

TẬP THỂ TÁC GIẢ:

PGS.TS. THÂN QUANG KHOÁT

PGS.TS. NGUYỄN THỊ KIM ANH

TS. ĐỖ TIẾN DŨNG

TS. NGÔ VĂN LINH

TS. NGUYỄN ĐỨC ANH

ĐƠN VỊ: KHOA KHOA HỌC MÁY TÍNH

TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HÀ NỘI, 2/2024

MỤC LỤC

MỞ ĐẦU **3**

CHƯƠNG 1. GIỚI THIỆU CHUNG VỀ HỌC MÁY VÀ KHAI PHÁ DỮ LIỆU **5**

1.1 Các khái niệm và bài toán học cơ bản 5

1.2 Quy trình xây dựng một hệ thống học 11

1.3 Vài vấn đề cần biết khi dùng ML 14

1.4 Thu thập và tiền xử lý dữ liệu 18

CHƯƠNG 2. HỒI QUI VÀ PHÂN CỤM **27**

2.1 Hồi qui 27

2.2 Phân cụm 35

CHƯƠNG 3. PHÂN LOẠI **58**

3.1 Học dựa trên láng giềng 58

3.2 Cây quyết định và Rừng ngẫu nhiên 65

3.3 Máy véctơ hỗ trợ (SVM) 75

3.4 Học dựa trên xác suất 88

3.5 Đánh giá hiệu năng và lựa chọn tham số 104

3.6 Mạng nơron nhân tạo 116

CHƯƠNG 4. KHAI PHÁ DỮ LIỆU **133**

4.1 Khai phá dữ liệu 133

4.2 Khai phá luật kết hợp 141

4.3 Mở rộng: Hiệu chỉnh 157

MỞ ĐẦU

Một trong những khát vọng lớn của ngành Trí tuệ nhân tạo (Artificial Intelligence - AI) là tạo ra các máy móc có khả năng hỗ trợ và sống hoà nhập với con người. Những máy móc ấy cần có khả năng hiểu, ứng dụng một cách đúng đắn tri thức và văn hoá của con người trong suốt thời gian sống của chúng. Đặc biệt, một khả năng không thể thiếu cho những máy móc ấy là khả năng học từ những quan sát về môi trường xung quanh.

Con người có thể học từ rất nhiều nguồn khác nhau, như sách, báo, tin tức, bạn bè, người thân, thầy, cô,... Chúng ta có thể học để làm những thứ đơn giản (như cầm đũa, đi lại, đạp xe, nấu cơm, giặt đồ,...), cho đến những thứ rất phức tạp (như lái xe ô tô, kinh doanh,...). Chúng ta có thể tự đúc kết kinh nghiệm sau khi đã quan sát nhiều lần về các sự vật hoặc hiện tượng. Đó là khả năng tự học rất tuyệt vời mà tự nhiên đã ban tặng.

Máy móc cũng có thể học như thế. Chúng có thể học được gì đó sau khi đã quan sát nhiều ví dụ khác nhau về một thứ gì đó. Đây là khả năng tự học từ các ví dụ (Learning from examples) hoặc quan sát (observations). Trong thực tế, các ví dụ này có thể thu thập được khá dễ dàng (thông qua camera, GPS, ăngten,...) và thường được gọi là Dữ liệu (Data). Máy có thể học tri thức mới từ dữ liệu đã thu thập được trong quá khứ, và từ đó có thể vận dụng tri thức đó để giải quyết các vấn đề trong tương lai, chẳng hạn như dự đoán, khám phá tri thức, lập kế hoạch. Khả năng này có thể giúp con người giải quyết nhiều thách thức trong thực tế.

Chúng ta đang sống trong kỷ nguyên lớn, nơi mà dữ liệu mới có thể được sinh ra hay được thu thập rất dễ dàng. Thông qua một số thiết bị hoặc phần mềm đơn giản, chúng ta có thể thu thập một lượng lớn dữ liệu từ một số nguồn (chẳng hạn Internet). Hơn thế nữa, những tiến bộ vượt bậc gần đây trong Học máy có thể giúp chúng ta sinh ra các tập dữ liệu theo mong muốn, bởi các mô hình sinh (Generative models). Do đó, một nhu cầu thực tế là làm sao có thể phát hiện ra được tri thức đang ẩn chứa trong lượng lớn dữ liệu đó. Tri thức ấy có thể hữu ích cho nhiều tác vụ khác trong doanh nghiệp, những người ra quyết định, hoạch định chính sách, ...

Trong khoá học này, chúng ta sẽ tìm hiểu những kiến thức căn bản nhất trong lĩnh vực Học máy và Khai phá dữ liệu. Các nội dung cụ thể sẽ được trình bày ở các chương

tiếp theo. Bố cục của tài liệu này bao gồm:

Chương 1 sẽ trình bày những khái niệm, bài toán cơ bản nhất, và một số vấn đề của các hệ thống có khả năng học.

Mục 1.4 trình bày về vấn đề thu thập dữ liệu và một số phương pháp tiền xử lý dữ liệu. Đây là một bước cơ bản nhưng cần thiết trong quá trình xây dựng một hệ thống có khả năng học hoặc khám phá tri thức.

Chương 2 bàn luận bài toán hồi qui và mô hình tuyến tính. Một số phương pháp huấn luyện khác nhau sẽ được trình bày, gồm bình phương tối thiểu, Ridge, và LASSO.

Mục 2.2 sẽ trình bày bài toán phân cụm, một ví dụ của lớp bài toán học không giám sát. Một số phương pháp phân cụm sẽ được trình bày, và một số vấn đề cũng sẽ được bàn luận chi tiết.

Chương 3 sẽ trình bày một số mô hình học máy để giải quyết bài toán phân loại, gồm K-NN, cây quyết định, rừng ngẫu nhiên, SVM, mô hình xác suất, Naive Bayes, mạng nơron nhân tạo. Chương này cũng bàn luận một số phương pháp để giúp ta đánh giá hiệu năng của một mô hình học máy, và việc lựa chọn siêu tham số cho chúng.

Chương 4 chứa các khái niệm và vấn đề cơ bản của Khai phá dữ liệu. Một ví dụ cụ thể là khai phá luật kết hợp sẽ được trình bày. Ngoài ra còn có thêm một số bàn luận về kỹ thuật hiệu chỉnh để giúp đối diện với vấn đề quá khớp trong học máy.

CHƯƠNG 1. GIỚI THIỆU CHUNG VỀ HỌC MÁY VÀ KHAI PHÁ DỮ LIỆU

1.1 Các khái niệm và bài toán học cơ bản

Đầu tiên, chúng ta hãy cùng làm quen một số khái niệm và bài toán cơ bản nhất.

1.1.1 Vài quan điểm về ML

Học (learning) là một khả năng căn bản của các hệ thống có trí thông minh thực sự. Do đó việc tạo khả năng học cho máy móc đã được đề ra và nghiên cứu khá lâu trước đây trong lĩnh vực AI. Tuy nhiên, chúng ta cần hiểu rõ học nghĩa là gì. Dưới đây là một quan điểm về việc học.

> "Learning denotes changes in the system that are adaptive in the sense that they enable the system to do the same task or tasks drawn from the same population more efficiently and more effectively the next time." [Simon, 1983]

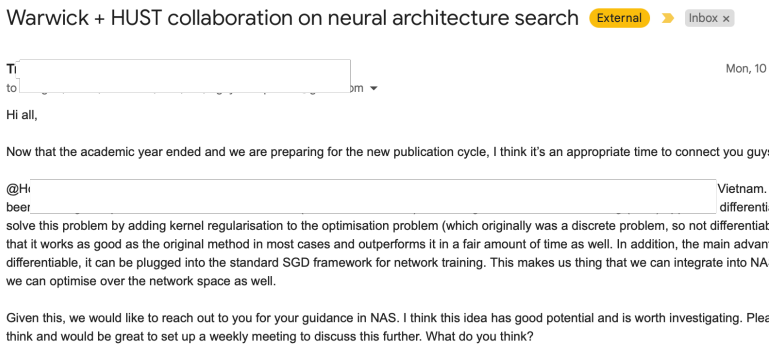
Quan điểm này cho ta một cái nhìn khá tổng quan về việc học, ở con người và các hệ thống máy móc. Khả năng học sẽ giúp các hệ thống ấy có khả năng thích nghi, tức là chúng có thể tự cải thiện hiệu năng khi giải quyết một nhiệm vụ (task) hoặc một lớp nhiệm vụ nào đó.

Quan điểm trên cũng có thể áp dụng cho máy để giúp ta hiểu về Học máy (Machine Learning - ML). Mặc dù vậy, cho đến nay có khá nhiều quan điểm (cách nhìn) khác nhau về ML. Dưới đây là một vài ví dụ:

> "Build systems that automatically improve their performance" [Simon, 1983]
>
> "Program computers to optimize a performance criterion using example data or past experience" [Alpaydin, 2020]

Dù được phát biểu ở nhiều góc nhìn khác nhau, nhưng chúng ta đều thấy một điểm chung là *"hiệu năng"* (performance). Nếu đứng ở góc độ ứng dụng thì chúng ta có thể xem: *ML là một lĩnh vực cung cấp một con đường để giải quyết một nhiệm vụ thực tế, thông qua khả năng tự cải thiện, dựa vào dữ liệu hoặc kinh nghiệm đã có.*

Cụ thể hơn, theo Mitchell [1997], một hệ thống **có khả năng học** nếu nó có khả năng tự cải thiện bản thân nó theo tiêu chí $\boldsymbol{P}$ khi giải quyết một nhiệm vụ $\boldsymbol{T}$, dựa vào dữ liệu


Warwick + HUST collaboration on neural architecture search External Inbox ×

Mon, 10

Hi all,

Now that the academic year ended and we are preparing for the new publication cycle, I think it's an appropriate time to connect you guy

Vietnam.

differenti

solve this problem by adding kernel regularisation to the optimisation problem (which originally was a discrete problem, so not differentiat that it works as good as the original method in most cases and outperforms it in a fair amount of time as well. In addition, the main advan differentiable, it can be plugged into the standard SGD framework for network training. This makes us thing that we can integrate into NA we can optimise over the network space as well.

Given this, we would like to reach out to you for your guidance in NAS. I think this idea has good potential and is worth investigating. Plea think and would be great to set up a weekly meeting to discuss this further. What do you think?

Hình 1: Ví dụ về một email.

hoặc kinh nghiệm $\boldsymbol{E}$. Một bài toán học máy có thể được mô tả bằng bộ ba $(\boldsymbol{T}, \boldsymbol{P}, \boldsymbol{E})$. Như vậy, đây là một quan điểm rất cụ thể về khả năng học của một hệ thống. Theo quan điểm này, chúng ta có thể đưa nhiều nhiệm vụ trong thực tế về các bài toán học máy. Dưới đây là vài ví dụ.

Ví dụ 1 (Lọc thư rác - Spam filtering) *Chúng ta muốn xây dựng một hệ thống có khả năng tự động lọc thư rác (spam) trong hòm thư điện tử (email). Hình 1 có ví dụ về một email.*

Có thể đưa nhu cầu này về bài toán học máy như sau:

- $\boldsymbol{T}$: Cần phán đoán một email cho trước có phải là thư rác hay không
- $\boldsymbol{P}$: Độ chính xác phán đoán (tỷ lệ emails được dự đoán đúng)
- $\boldsymbol{E}$: Tập hợp các email trong quá khứ, mỗi email đã được gán nhãn (rác/thường) tương ứng.

Ví dụ 2 (Mô tả ảnh - Image captioning) *Chúng ta muốn xây dựng một hệ thống có khả năng tự động đưa ra mô tả nội dung bên trong bức ảnh.*

Có thể đưa nhu cầu này về bài toán học máy như sau:

- $\boldsymbol{T}$: Cần đưa ra mô tả ngắn gọn về nội dung bên trong một bức ảnh cho trước
- $\boldsymbol{P}$: Độ chính xác của mô tả
- $\boldsymbol{E}$: Tập hợp các bức ảnh, mỗi bức ảnh đã được mô tả bởi một câu ngắn gọn về nội dung bên trong bức ảnh đó. Hình 2 chứa một tập ví dụ.[1]

[1] Chúng được lấy từ https://openai.com/dall-e-3.



lychee-inspired spherical chair



a girl giving cat a gentle hug



a small hedgehog holding a piece of watermelon

Hình 2: Tập các bức ảnh và các mô tả về chúng.

1.1.2 Bản chất của việc học, suy diễn, và phán đoán

Con người có khả năng học rất tuyệt vời. Chúng ta có thể học từ những thứ đơn giản đến những thứ rất phức tạp (ví dụ như lái xe). Chúng ta có thể tự đúc rút ra kinh nghiệm của riêng mình khi đã quan sát nhiều lần, và về sau chúng ta có thể sử dụng kinh nghiệm ấy để giải quyết một công việc nào đó (ví dụ, lái xe). Như vậy, mỗi khi học, bộ não của chúng ta sẽ nhận nhiều tín hiệu (thông qua các quan sát bằng mắt, tai, ...) từ bên ngoài, rồi thực hiện xử lý hoặc tính toán nào đó, để thu được tri thức hoặc kinh nghiệm. Những tri thức này có thể được dùng để giải quyết một nhiệm vụ (trong tương lai).

Như vậy, bản chất của mỗi quá trình học là tìm một hàm y^* nào đó mà kết nối mỗi đầu vào $\boldsymbol{x}$ với đầu ra y. Nghĩa là quá trình học sẽ đi tìm một hàm (chưa biết) có dạng:

$$y^* : \boldsymbol{x} \mapsto y \tag{1}$$

trong đó:

- $\boldsymbol{x}$ là một quan sát (observation) hoặc tri thức cũ. Mỗi quan sát thường ở dưới dạng một tập tín hiệu và thường được gọi là một *mẫu dữ liệu* (data instance) hoặc một *ví dụ* (example).
- y có thể là tri thức mới hoặc kinh nghiệm mới. Đôi khi nó là một phán đoán (prediction).

Trong thực tế, chúng ta thường không biết hàm y^*, về hình dạng hay tính chất của nó. Nhưng đôi khi chúng ta có thể thu thập được một số quan sát $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N\}$ về đầu vào $\boldsymbol{x}$ và một số đầu ra $\{y_1, y_2, \ldots, y_M\}$ tương ứng, tức là $y_i = y^*(\boldsymbol{x}_i)$ đối với vài mẫu thứ i. Máy sẽ cần tìm ra hàm y^* từ một tập dữ liệu $\boldsymbol{D} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N, y_1, y_2, \ldots, y_M\}$ đã thu

thập trong quá khứ. Tập $\boldsymbol{D}$ thường được gọi là **tập huấn luyện (training set)** hoặc *tập học*.

Định nghĩa 1 (Bài toán học - Learning problem) *Cần tìm ra hàm* $y^* : \boldsymbol{x} \mapsto y$ *từ một tập dữ liệu* $\boldsymbol{D} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, ..., \boldsymbol{x}_N, y_1, y_2, ..., y_M\}$ *cho trước.*[2]

trong đó:

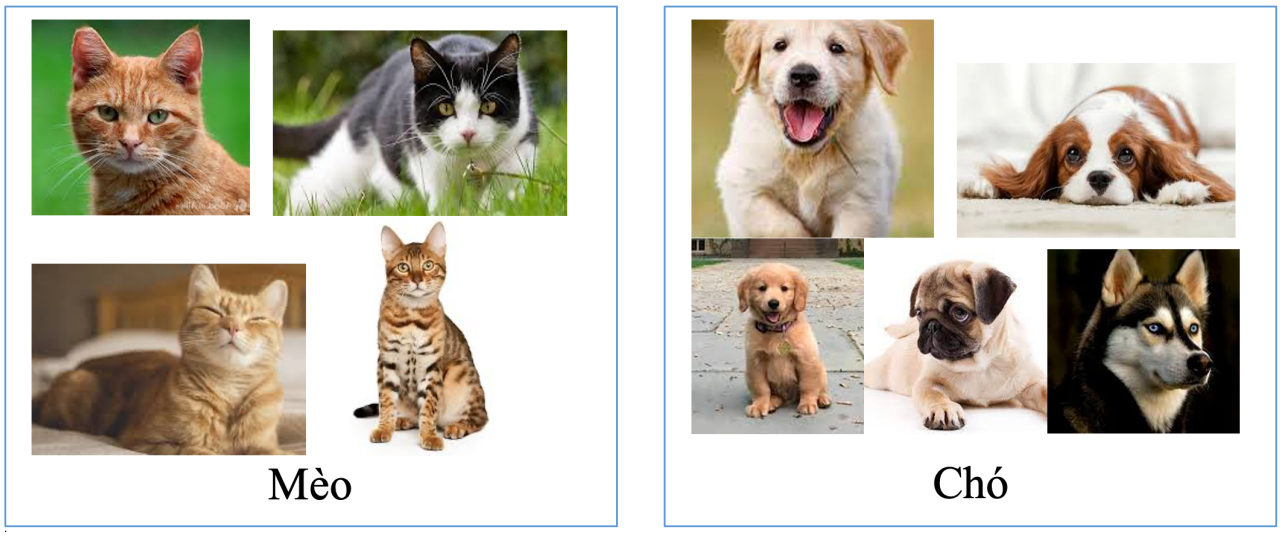
- Mỗi $\boldsymbol{x}_i$ là một quan sát cụ thể về đầu vào $\boldsymbol{x}$.
- y_j là một quan sát về y, sao cho $y_j = y^*(\boldsymbol{x}_j)$. Chúng thường được gọi là *nhãn* (label) hoặc *phản hồi* (response) hoặc *đầu ra* (output).
- $\boldsymbol{x}$ thường thuộc một không gian $\mathscr{X}$ nào đó và thường được gọi là *không gian dữ liệu* (data space). Nghĩa là $\boldsymbol{x} \in \mathscr{X}$. Còn y thường thuộc một tập $\mathscr{Y}$ hữu hạn hoặc vô hạn. $\mathscr{Y}$ có thể được gọi là *không gian đầu ra* hoặc *tập nhãn*.
- Thông thường $N \geq M$, nghĩa là số lượng nhãn thu thập được thường ít hơn số mẫu đầu vào.

Có thể nói rằng bài toán tìm ra hàm y^* từ một tập dữ liệu $\boldsymbol{D}$ là một thách thức lớn nếu không có thông tin thêm về hàm đó. Một lý do quan trọng là không gian tất cả các hàm quá lớn. Do đó đây là một bài toán bất khả thi.

Một cách làm khả thi và phổ biến trong ML là sử dụng một lớp hàm $\mathscr{H}$ mà có thể dễ dàng làm việc, rồi tìm ra một hàm $f \in \mathscr{H}$ để xấp xỉ hàm y^*. Một **thuật toán học** (learning algorithm) sẽ đi tìm hàm f này khi cho trước tập $\boldsymbol{D}$ và lớp hàm $\mathscr{H}$. Thông thường chúng ta hy vọng rằng hàm f sẽ xấp xỉ tốt hàm y^* tại mọi điểm, nghĩa là $f(\boldsymbol{x}) \approx y^*(\boldsymbol{x}), \forall \boldsymbol{x} \in \mathscr{X}$.

Sau khi học, chúng ta thu được hàm f. Hàm này có thể chứa tri thức hoặc kinh nghiệm mới cho máy. Đôi khi nó cũng được gọi là *mô hình* (model). Chúng ta có thể dùng hàm này để đưa ra dự đoán nhãn $f(\boldsymbol{z})$ cho mỗi mẫu dữ liệu $\boldsymbol{z}$ trong tương lai. Giai đoạn này thường được gọi là **phán đoán** (predict) hoặc **suy diễn** (infer).

[2]Trong một số ứng dụng thực tế, hàm y^* có thể ở dạng phức tạp hơn nhiều. Ví dụ hàm thay đổi theo thời gian, hàm của biến ngẫu nhiên $\boldsymbol{x}$, hàm có đầu ra dạng cấu trúc, ... Tuy nhiên, ở môn học này chúng ta sẽ xét dạng cơ bản nhất để dễ dàng nắm bắt các khái niệm và một số vấn đề quan trọng trong ML.



Hình 3: Ví dụ về một tập dữ liệu đã được gắn nhãn. Mỗi ảnh là một mẫu dữ liệu, và nhãn tương ứng ("mèo" hoặc "chó").

1.1.3 Hai bài toán học cơ bản

Như đã bàn luận ở trên, bài toán học về cơ bản là khó. Để dễ dàng làm việc hơn thì người ta thường chia thành các bài toán nhỏ hơn. Tiếp theo, chúng ta sẽ tìm hiểu hai bài toán con.

Định nghĩa 2 (Học có giám sát - Supervised learning) *Cần tìm hàm* $y^* : \boldsymbol{x} \mapsto y$ *từ một tập dữ liệu* $\boldsymbol{D} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, ..., \boldsymbol{x}_M, y_1, y_2, ..., y_M\}$ *cho trước, trong đó* $y_j = y^*(\boldsymbol{x}_j)$ *với mỗi chỉ số* $j \in \{1, 2, ..., M\}$.

Trong bài toán này, mỗi mẫu dữ liệu $\boldsymbol{x}_j$ trong tập học có một nhãn y_j tương ứng. Và như thế, tập $\boldsymbol{D}$ chứa M cặp $(\boldsymbol{x}_j, y_j)$. Ở đây y_j có thể coi là thông tin để hướng dẫn (supervision) việc tìm ra hàm y^*. Trong thực tế đôi khi chúng ta có thể thu thập được những nhãn này khá dễ dàng, chẳng hạn nhãn spam cho một email, tên con vật có trong một bức ảnh. Hình 3 cho ví dụ về một tập dữ liệu đã có nhãn.

Tuỳ thuộc vào miền $\mathscr{Y}$ của đầu ra y, chúng ta có thể làm việc với các bài toán nhỏ hơn, như

- **Phân loại (classification):** nếu miền $\mathscr{Y}$ rời rạc và hữu hạn. Ví dụ $\mathscr{Y} = \{\text{thường, rác}\}$, $\mathscr{Y} = \{\text{mèo, chó}\}$, $\mathscr{Y} = \{\text{thích, yêu, ghét, giận}\}$. Hình 3 mô tả một tập huấn luyện cho bài toán phân loại với $\mathscr{Y} = \{\text{mèo, chó}\}$.
- **Hồi qui (regression):** nếu $\mathscr{Y} \subseteq \mathbb{R}$, tức là miền số thực

Như vậy đối với bài toán phân loại, chúng ta thường có một tập nhãn lớp $\mathscr{Y}$ chọn trước và cố định. Nếu mỗi mẫu $\boldsymbol{x}$ có một $y \in \mathscr{Y}$ (tức là chỉ nhận một giá trị trong $\mathscr{Y}$)

thì chúng ta đang làm việc với bài toán *Phân loại nhiều lớp* (multiclass classification). Nếu tập $\mathscr{Y}$ chỉ có 2 phần tử duy nhất thì đôi khi người ta gọi đó là bài toán *Phân loại nhị phân* (binary classification). Trong trường hợp nhãn y cho mỗi mẫu dữ liệu $\boldsymbol{x}$ có thể nhận vài giá trị trong $\mathscr{Y}$ thì chúng ta đang làm việc với bài toán *Phân loại đa nhãn* (multilabel classification).

Ứng dụng: có rất nhiều nhu cầu trong thực tế có thể đưa về một bài toán học có giám sát, chẳng hạn:

- Phân loại: lọc email, dự đoán mức độ rủi ro của một hồ sơ vay tín dụng, dự đoán bất thường trong giao dịch tài khoản ngân hàng, ...
- Phân loại đa nhãn: phân tích quan điểm trong một đoạn văn, gán tag cho ảnh, ...
- Hồi qui: dự đoán chỉ số chứng khoán cho mỗi phiên giao dịch, dự báo lượng mưa trong ngày, dự báo độ ẩm không khí, dự báo giá nhà, ...

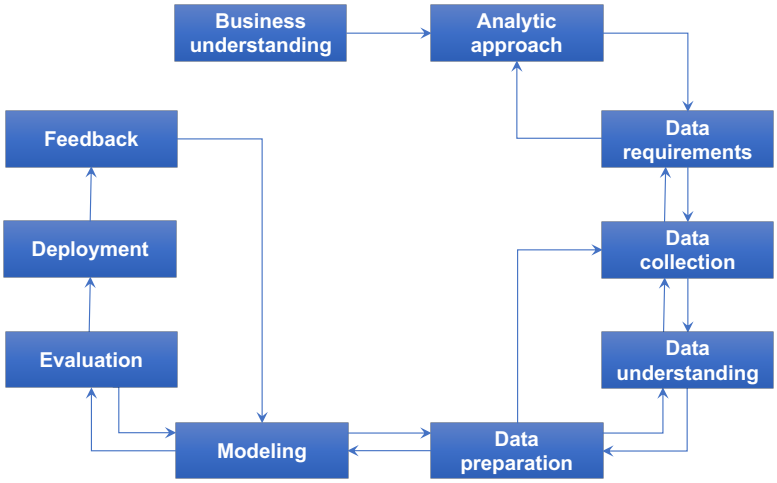
Định nghĩa 3 (Học không giám sát - Unsupervised learning) *Tìm hàm* $y^* : \boldsymbol{x} \mapsto y$ *từ một tập dữ liệu* $\boldsymbol{D} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, ..., \boldsymbol{x}_M\}$ *cho trước.*

Có rất nhiều nhu cầu trong thực tế có thể đưa về bài toán này. Chẳng hạn, chúng ta muốn gom các bình luận của người dùng khác nhau về các nhóm nhỏ để xem họ đang nói về những gì; chúng ta muốn phát hiện các tương tác ẩn giữa các nhà chính trị gia thông qua tin tức; chúng ta muốn phát hiện xu hướng (thị hiếu) gần đây của người dùng thông qua các trao đổi (comments, posts) trên Facebook, ...

Dễ dàng nhận thấy, trong bài toán học không giám sát, tập huấn luyện không có nhãn cho các mẫu dữ liệu. Mặc dù các mẫu dữ liệu có thể thu thập dễ, nhưng việc thu thập nhãn cho chúng đôi khi rất tốn kém hoặc phức tạp. Ví dụ, "xu hướng" là thứ rất khó thu thập đối với nhiều người, do khả năng phát hiện xu hướng bị hạn chế. Trong mục 2.2, chúng ta sẽ làm quen với một ví dụ cơ bản về bài toán này.

1.1.4 Phân biệt giữa Học máy và Khai phá dữ liệu

Ở phần trước chúng ta đã thấy rằng ML có thể giúp tạo ra một hệ thống mà có thể học từ dữ liệu để tìm ra tri thức mới. Khả năng tìm ra tri thức này có liên hệ chặt chẽ với một lĩnh vực có tên gọi là **Khai phá dữ liệu** (Data mining - DM) [Han et al., 2023].



Hình 4: Quy trình xây dựng một hệ thống dựa trên học máy.

Mối quan tâm chính của DM là việc tìm ra những tri thức (knowledge) hoặc mẫu (patterns) ẩn trong các tập dữ liệu lớn, bằng việc sử dụng các thuật toán cụ thể nào đó một cách (bán) tự động. Những tri thức này thường không tường minh, ở nhiều hình dạng khác nhau, và có thể hữu ích trong tương lai. DM là một bước chính trong toàn bộ quy trình về phát hiện tri thức (KDD) [Fayyad et al., 1996].

Như vậy có một mối liên hệ mật thiết giữa ML và DM. Mỗi thuật toán học thường cần tìm ra tri thức từ một tập dữ liệu cho trước. Do đó các thuật toán ML hay được sử dụng cho DM hay KDD. Tuy nhiên "học" chưa thể coi là khám phá tri thức [Simon, 1983], bởi việc học đôi khi chỉ thu được những tri thức đã sẵn có từ người hoặc hệ thống khác (ví dụ học lái xe, học viết, ...). Trong khi đó, KDD thường muốn tìm ra những tri thức mới mẻ và có tiềm năng hữu ích.

1.2 Quy trình xây dựng một hệ thống học

Để xây dựng một hệ thống có khả năng học, chúng ta cần thực hiện những bước chính như trong Hình 4.[3] Cụ thể như sau:

1. *Business understanding:* Đầu tiên chúng ta cần hiểu rõ nhu cầu thực tế đang được đặt ra để giải quyết. Đôi khi nhu cầu đó có thể nằm ngoài lĩnh vực hiểu biết của chúng ta. Cho nên cần tìm hiểu rõ nó là gì.

2. *Analytic approach:* Chúng ta cần đưa nhu cầu trên về một bài toán ML. Ở bước này, chúng ta cần phát biểu bài toán ML rõ ràng, phù hợp để giải quyết nhu cầu

[3]Một số bước có tham khảo từ http://www.theta.co.nz.

thực tế kia. Chú ý rằng, một nhu cầu thực tế có thể đưa về nhiều bài toán ML khác nhau. Ví dụ, nhu cầu "chuẩn đoán bệnh viêm gan" có thể đưa về bài toán phân loại, nhưng cũng có thể đưa về một bài toán ML khác; nhu cầu "dự đoán giá nhà" có thể đưa về bài toán hồi qui, nhưng cũng có thể đưa về bài toán phân loại. Lựa chọn ở bước này sẽ quyết định trực tiếp bước sau.

3. *Data requirements:* Tiếp theo chúng ta cần xác định rõ các yêu cầu về tập dữ liệu cần thu thập để huấn luyện hệ thống, tuỳ vào bài toán ML đã chọn ở trên. Ví dụ, nếu ở trên ta chọn bài toán hồi qui thì mỗi mẫu dữ liệu huấn luyện cần có nhãn. Nhãn là yêu cầu bắt buộc trong trường hợp này. Ngoài ra có thể đưa thêm một số yêu cầu khác.

4. *Data collection:* Sau khi đã biết các yêu cầu về dữ liệu, chúng ta cần xây dựng một hệ thống (bán) tự động hoặc cử người để thu thập tập dữ liệu huấn luyện. Chú ý rằng, nếu thấy có vấn đề gì đó trong việc thu thập dữ liệu, chẳng hạn quá đắt đỏ, thì cần xem xét chỉnh sửa hai bước trên.

5. *Data understanding:* Sau khi có một tập dữ liệu, chúng ta có thể thực hiện một vài tính toán đơn giản hoặc dùng các công cụ trực quan hoá để xem xét một vài tính chất của dữ liệu. Ví dụ, xem sự đa dạng của một thuộc tính, mức độ trống/thiếu giá trị trong các mẫu dữ liệu, ... Sau bước này, chúng ta có thể thu được một vài hiểu biết về đặc trưng của dữ liệu trong bài toán này. Chúng có thể hữu ích cho các bước sau.

6. *Data preparation:* Tiếp đó, chúng ta cần thực hiện tiền xử lý dữ liệu. Tập dữ liệu thu thập được thường ở dạng thô. Do đó cần đưa chúng về dạng phù hợp hơn với các mô hình ML. Có thể thực hiện một vài bước nhỏ, chẳng hạn lọc nhiễu, loại bỏ lỗi, biến đổi về dạng vectơ, ... Chương 1.4 sẽ bàn luận kỹ hơn.

7. *Modeling:* Sau khi có tập dữ liệu đã được tiền xử lý $\boldsymbol{D}$, chúng ta chọn một mô hình ML phù hợp và huấn luyện nó từ $\boldsymbol{D}$.

8. *Evaluation:* Sau khi có mô hình vừa huấn luyện, chúng ta cần tiến hành đánh giá hiệu năng (chất lượng) của nó. Cần dùng chiến lược và độ đo phù hợp để đánh giá. Nếu thấy mô hình chưa tốt, chúng ta có thể quay lại những bước trước để kiểm tra hoặc chỉnh sửa. Ví dụ, kiểm tra việc huấn luyện đã thành công chưa, tham số đã

tốt chưa, mô hình có phù hợp với bài toán không, ... Đôi khi cần kiểm tra việc tiền xử lý, hoặc quá trình thu thập dữ liệu.

9. *Deployment:* Nếu mô hình được huấn luyện tốt thì có thể dùng nó để đưa vào triển khai hệ thống (sản phẩm) đầy đủ.

10. *Feedback:* Hệ thống sau khi hoàn thành có thể cần người dùng đánh giá. Ở bước này, chúng ta thu thập các phản hồi của người dùng về hệ thống. Nếu các phản hồi cho thấy hệ thống chưa đủ tốt thì có thể quay trở lại các bước phía trước để cải thiện. Ngược lại thì xem như hệ thống đạt yêu cầu và việc xây dựng đã thành công.

Đối với nhu cầu khai phá dữ liệu, một số bước ở trên có thể sử dụng. Chi tiết sẽ được trình bày trong Chương 4.

1.2.1 Hàm cần học và tập huấn luyện

Ở bước 2 trong quy trình trên, chúng ta cần đưa nhu cầu thực tế về một bài toán ML. Nghĩa là chúng ta cần xác định rõ hàm cần học $y^* : \mathscr{X} \to \mathscr{Y}$ có đặc trưng cơ bản ra sao về đầu vào và đầu ra. Nói cách khác, ta cần xác định được không gian dữ liệu $\mathscr{X}$ và tập nhãn $\mathscr{Y}$. Ví dụ:

- Bài toán lọc thư rác: $\mathscr{X}$ là không gian tất cả thư điện tử (emails); $\mathscr{Y} = \{0, 1\}$, trong đó nhãn 1 nghĩa là rác, 0 là thư thường
- Bài toán phân biệt loài vật: $\mathscr{X}$ là không gian tất cả những bức ảnh RGB có kích cỡ 1024×1024, $\mathscr{Y} = \{dog, cat, elephant\}$

Đối với việc thu thập dữ liệu: ta cần thiết kế kỹ lưỡng bởi dữ liệu thu thập được sẽ dùng để huấn luyện hệ thống. Chất lượng (hay hiệu năng) của một hệ thống phụ thuộc rất lớn vào tập dữ liệu này. Để hệ thống có hiệu quả cao, tập huấn luyện cần chứa những mẫu mà có thể mô tả được nhiều đặc trưng cốt lõi của không gian $\mathscr{X}$. Nghĩa là, tập dữ liệu này cần có tính đại diện cao cho không gian $\mathscr{X}$. Ngược lại, tập dữ liệu đó có thể chứa quá ít thông tin về $\mathscr{X}$ và sẽ làm cho hệ thống làm việc kém với những mẫu dữ liệu trong tương lai.

1.2.2 Giai đoạn Mô hình hoá

Ở giai đoạn mô hình hoá (Modeling), có hai bước quan trọng cần làm: chọn kiểu mô hình và giải thuật học.

a) Chọn kiểu mô hình:

Vì hàm y^* thường không biết và ta chỉ thu thập được tập dữ liệu $\boldsymbol{D}$, nên một cách phổ biến trong ML là chọn một kiểu (lớp) mô hình $\mathscr{H}$ để xấp xỉ hàm y^*. Lớp mô hình này thường ở dạng đã biết và dễ làm việc. Ví dụ, mô hình tuyến tính, mạng nơron, cây quyết định, ...

Như vậy $\mathscr{H}$ là một lớp các hàm, mà mỗi hàm còn được gọi là *giả thuyết* (hypothesis). Khi chọn $\mathscr{H}$, ta đã ngầm giả thuyết rằng trong $\mathscr{H}$ sẽ chứa một hàm f mà có thể xấp xỉ tốt y^* tại mọi điểm, nghĩa là $f(\boldsymbol{x}) \approx y^*(\boldsymbol{x}), \forall \boldsymbol{x} \in \mathscr{X}$.

b) Chọn giải thuật học:

Tiếp theo ta cần chọn một **giải thuật học** A nào đó để giúp ta tìm ra một hàm $f \in \mathscr{H}$, dựa vào tập học $\boldsymbol{D}$. Nói cách khác, một giải thuật học A sẽ trả về một hàm f khi cho trước tập học $\boldsymbol{D}$ và lớp mô hình $\mathscr{H}$. Khi đó ta có thể viết $f = \mathsf{A}(\mathscr{H}, \boldsymbol{D})$. Hàm f này đôi khi còn được gọi là *mô hình đã được huấn luyện* (trained model).

Trong ML, mỗi kiểu mô hình khác nhau có thể có phương pháp học khác nhau. Ví dụ phương pháp Lan truyền ngược (backpropagation) sẽ huấn luyện một mạng nơron, phương pháp ID3 sẽ huấn luyện một cây quyết định, ... Chúng ta sẽ làm quen với một vài giải thuật học ở các chương tiếp theo.

1.3 Vài vấn đề cần biết khi dùng ML

Trong phần này, chúng ta sẽ bàn luận đến mục tiêu quan trọng của ML và một số vấn đề liên quan đến việc học.

1.3.1 Thuận toán học (Learning algorithm)

Đối với nhiều mô hình học máy, một thuật toán học A có thể ở dạng lặp. Mỗi bước lặp có thể tìm ra một hàm nào đó trong $\mathscr{H}$, bước lặp sau cố gắng cải thiện bước lặp trước. Tuy nhiên nếu không cẩn thận thì thuật toán có thể không hội tụ tới một hàm tốt. Khi đó mô hình chưa được huấn luyện tốt.

Đối với một bài toán học đã cho, có nhiều thuật toán học khác nhau để lựa chọn. Ví dụ, có nhiều giải thuật khác nhau để huấn luyện mạng nơron [Goodfellow et al., 2016]. Mỗi thuật toán khác nhau có thể cho kết quả khác nhau. Đó là đặc điểm quan trọng trong ML. Lúc đó, một câu hỏi là thuật toán nào sẽ tốt nhất đối với bài toán ta đang có?

Định lý "No Free Lunch" [Wolpert and Macready, 1997] nói rằng *nếu một thuật toán hiệu quả đối với một lớp bài toán này thì nó sẽ trả giá về hiệu quả đối với tập các bài toán còn lại*.

Định lý này gợi rằng không có thuật toán nào luôn hiệu quả nhất trên mọi miền ứng dụng. Đây là một đặc trưng quan trọng trong ML và phân tích dữ liệu. Do đó khi đối diện với mỗi bài toán, chúng ta cần lựa chọn một giải thuật học (và mô hình) phù hợp với bài toán đó.

1.3.2 Tập huấn luyện (Training set)

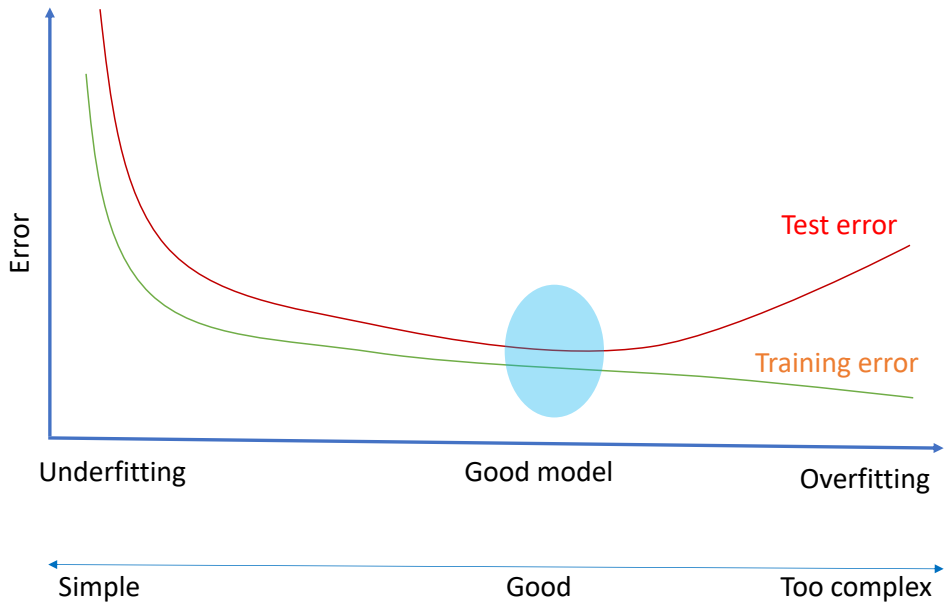
Tập huấn luyện là nơi chứa tri thức để cho máy học và quyết định chất lượng tri thức của máy. Một tập học tốt có thể cung cấp nhiều tri thức hữu ích cho máy. Do đó tính đại diện của các mẫu dữ liệu trong tập học đóng vai trò quan trọng.

Trong ML có một đặc trưng thường thấy là các mô hình có thể có chất lượng càng tốt hơn nếu được huấn luyện từ nhiều mẫu dữ liệu hơn. Nói cách khác, việc tăng kích cỡ tập học lên thì thường giúp mô hình có thể tốt hơn. Tuy nhiên, điều này sẽ dẫn đến một câu hỏi là "kích cỡ tập huấn luyện như thế nào là đủ?" Việc trả lời câu hỏi này sẽ cần một số kiến thức về lý thuyết học [Mohri et al., 2018].

Trong nhiều miền ứng dụng, dữ liệu thường bị nhiễu, lỗi, thiếu, ... Những yếu tố này thường ảnh hưởng mạnh mẽ đến hiệu quả của các mô hình ML. Nếu bị nhiễu nhiều, tập học có thể tạo ra hướng dẫn sai lệch cho giải thuật học, và do đó tri thức học được có thể không tốt. Chúng ta cần tìm cách vượt qua những tình huống khó này.

1.3.3 Khả năng tổng quát hoá (Generalization)

Một trong những mục tiêu quan trọng nhất của các mô hình ML là **Khả năng tổng quát hoá (Generalization)**, tức là sau khi học, mô hình có thể làm việc (phán đoán) tốt với những mẫu dữ liệu sẽ gặp trong tương lai. Như vậy khi có khả năng tổng quát hoá cao, mô hình có thể làm việc hiệu quả đối với những mẫu dữ liệu không có trong



Hình 5: Minh họa về quá khớp (Overfitting) và kém khớp (Underfitting). "Test error" mô tả lỗi phán đoán của mô hình đối với những mẫu dữ liệu không có mặt trong quá trình huấn luyện mô hình ấy.

tập huấn luyện. Những mô hình như thế sẽ giúp những hệ thống ML mang lại lợi ích cho nhiều ứng dụng thực tế.

Ngược lại, nếu khả năng tổng quát hoá kém, mô hình ML có thể đang bị vướng phải một số vấn đề, chẳng hạn Quá khớp hoặc kém khớp. Trong những trường hợp đó, chúng có thể không có ích.

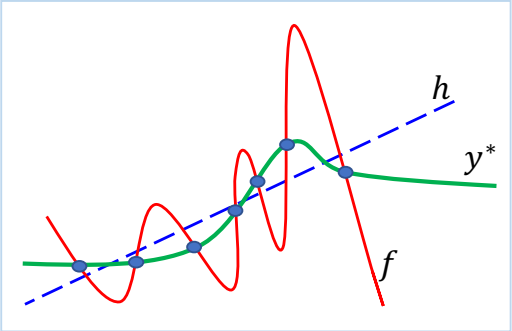
1.3.4 Quá khớp (Overfitting) và kém khớp (Underfitting)

Đôi khi trong thực tế chúng ta có thể gặp phải tình huống: một mô hình đạt độ chính xác cao trên tập huấn luyện $\boldsymbol{D}$, nhưng phán đoán kém chính xác với các mẫu dữ liệu bên ngoài $\boldsymbol{D}$. Tình huống này thường được gọi là **Quá khớp (Overfitting)**.

Theo Mitchell [1997], một hàm f được gọi là quá khớp nếu tồn tại hàm g sao cho: g có thể tệ hơn f khi phán đoán các mẫu huấn luyện $\boldsymbol{D}$, nhưng g tốt hơn f đối với các mẫu dữ liệu ngoài $\boldsymbol{D}$.

Hình 5 mô tả một vài tình huống có thể gặp khi dùng một mô hình ML. Có nhiều nguyên nhân dẫn đến vấn đề quá khớp. Chẳng hạn,

- *Mô hình quá phức tạp:* nghĩa là mỗi hàm trong lớp mô hình $\mathscr{H}$ quá phức tạp. Khi đó có nhiều hàm $f \in \mathscr{H}$ dễ dàng khớp hoàn hảo với một tập huấn luyện. Hình 6 chứa một ví dụ về một hàm f phức tạp (màu đỏ) mà có thể xấp xỉ tập học một cách hoàn hảo. Nhưng f xấp xỉ y^* rất kém về tổng thể.



Hình 6: Ví dụ về một số hàm khi xấp xỉ một tập dữ liệu (các dấu chấm). y^* là hàm đúng (ground truth), nhưng ta thường không biết.

- *Nhiễu hoặc lỗi trong dữ liệu:* nhiễu hoặc lỗi có thể làm sai lệch thông tin ẩn chứa trong các mẫu dữ liệu. Do đó thuật toán học dễ bị đánh lừa và tìm ra hàm không tốt trong $\mathscr{H}$.
- *Tập học quá bé:* khi đó một mô hình dễ dàng có thể xấp xỉ tập học một cách hoàn hảo. Tuy nhiên do kích cỡ bé nên tập dữ liệu này chưa chắc đã đại diện tốt cho toàn bộ không gian dữ liệu $\mathscr{X}$ và đặc trưng của hàm y^*. Nghĩa là tập học đó có thể cung cấp rất ít tri thức về $\mathscr{X}$ và hàm y^*.

Một tình huống khác là **Kém khớp** (Underfitting). Khi đó một hàm kém khớp h thường có lỗi phán đoán lớn (về mặt trung bình) cho bất kỳ mẫu dữ liệu nào, ngay cả những mẫu đã học. Hình 5 minh họa tình huống kém khớp. Như vậy h đã không được huấn luyện đến nơi đến chốn hoặc là hàm quá đơn giản so với y^*.

Trường hợp kém khớp có thể diễn ra nếu chúng ta dùng một kiểu mô hình $\mathscr{H}$ quá đơn giản để giải quyết một bài toán phức tạp. Khi đó bất kỳ hàm nào trong $\mathscr{H}$ đều xấp xỉ y^* kém. Xét ví dụ hàm y^* trong Hình 6: nếu chúng ta chọn $\mathscr{H}$ là lớp mô hình dạng tuyến tính (các đường thẳng) thì vấn đề kém khớp sẽ diễn ra. Hàm $h \in \mathscr{H}$ (đường nét đứt, màu xanh) sẽ luôn có độ chính xác kém tại phần lớn các mẫu dữ liệu.

1.3.5 Hiệu chỉnh (regularization)

Chúng ta đã biết rằng khả năng tổng quát hoá cao mà một mục tiêu quan trọng của các hệ thống có khả năng học. Nghĩa là khả năng phán đoán trong tương lai phải có độ chính xác cao. Nhưng nếu quá khớp hay kém khớp diễn ra thì hệ thống không đạt được mục tiêu này. Do đó khi xây dựng những hệ thống học (hay khi huấn luyện một mô hình) thì cần có giải pháp để tránh những vấn đề đó.

Đối với kém khớp, chúng ta có thể lựa chọn kiểu mô hình mới mà có khả năng mạnh hơn, chứa các hàm phức tạp hơn. Ví dụ, thay vì chọn mô hình tuyến tính, chúng ta chọn mô hình cây quyết định hoặc mạng nơron. Ngoài ra, chúng ta cũng cần kiểm tra việc huấn luyện và đảm bào rằng quá trình huấn luyện đã thành công và tốt.

Đối với quá khớp, một con đường thường được sử dụng trong ML là **Hiệu chỉnh** (regularization). Đây là cách giúp ta giảm bớt vấn đề quá khớp cho các mô hình ML. Cho đến nay, có rất nhiều phương pháp hiệu chỉnh khác nhau đã được đề ra. Nói một cách dễ hiểu thì các phương pháp hiệu chỉnh thường thu hẹp không gian tìm kiếm. Nghĩa là quá trình học sẽ tìm kiếm một hàm f trong một vùng nhỏ $\mathscr{H}' \subset \mathscr{H}$. Mỗi cách hiệu chỉnh sẽ tạo ra vùng $\mathscr{H}'$ khác nhau. Chúng ta sẽ tìm hiểu chi tiết hơn về một số phương pháp hiệu chỉnh ở Chương 2.

1.4 Thu thập và tiền xử lý dữ liệu

Thu thập (data collection) và tiền xử lý dữ liệu (data preprocessing) là hai bước quan trọng trong quá trình xử lý thông tin hiện đại. Việc thu thập dữ liệu đòi hỏi sự tập trung và tổ chức để lấy được thông tin cần thiết từ nhiều nguồn khác nhau, bao gồm cả cơ sở dữ liệu, bảng tính, thiết bị cảm biến, và nguồn thông tin trực tuyến. Quá trình này giúp xây dựng cơ sở dữ liệu đầy đủ và đa dạng, làm nền tảng cho các phân tích và quyết định sau này.

Sau khi dữ liệu được thu thập, quá trình tiền xử lý trở thành bước quan trọng để làm cho dữ liệu trở nên hữu ích và chính xác hơn. Tiền xử lý dữ liệu bao gồm loạt các bước: Làm sạch dữ liệu (data cleaning) (lọc dữ liệu nhiễu, điền giá trị còn thiếu, chuẩn hóa đơn vị đo, và kiểm tra tính nhất quán của dữ liệu), tích hợp dữ liệu (data integration) và biến đổi dữ liệu (data transformation). Mục tiêu là tạo ra một bộ dữ liệu có chất lượng cao, sẵn sàng cho các phân tích và mô hình hóa dữ liệu trong bước khai phá dữ liệu về sau. Quá trình này không chỉ giúp cải thiện độ chính xác của thông tin mà còn làm giảm thời gian và công sức cần thiết cho các công đoạn xử lý dữ liệu tiếp theo. Sự kết hợp chặt chẽ giữa thu thập và tiền xử lý dữ liệu đóng vai trò quan trọng trong việc đảm bảo rằng dữ liệu được sử dụng là đáng tin cậy và có giá trị trong quyết định và phân tích.

1.4.1 Thu thập dữ liệu

Thu thập dữ liệu là quá trình tổng hợp dữ liệu từ nhiều nguồn để tìm kiếm câu trả lời cho các vấn đề nghiên cứu, giải đáp câu hỏi, đánh giá kết quả và dự đoán xu hướng và khả năng xảy ra. Đây là một pha quan trọng trong tất cả các loại nghiên cứu, phân tích và quyết định, bao gồm cả những nghiên cứu trong các ngành khoa học xã hội, doanh nghiệp và chăm sóc sức khỏe. Trong quá trình thu thập dữ liệu, chúng ta phải xác định loại dữ liệu, nguồn dữ liệu và phương pháp đang được sử dụng. Có nhiều phương pháp thu thập dữ liệu khác nhau. Lựa chọn cách thu thập dữ liệu phụ thuộc vào từng bài toán.

Quá trình thu thập dữ liệu, lấy mẫu (sampling) thường xuyên được thực hiện. Lấy mẫu là quá trình chọn lựa một phần của toàn bộ dữ liệu để đại diện cho toàn bộ tập dữ liệu này. Lấy mẫu giúp chi phí và tài nguyên tính toán khi tập dữ liệu là rất lớn. Phương pháp lấy mẫu phải có tính đại diện và hiệu quả. Tính đại diện hướng tới lấy mẫu đại diện cho đặc điểm của quần thể mà chúng ta đang nghiên cứu. Để làm được điều này, mẫu lấy được phải đa dạng và tránh bị thiên lệch. Trong khi đó, tính hiệu quả hướng tới tiết kiệm thời gian, chi phí và nỗ lực so với việc lấy dữ liệu từ toàn bộ tập dữ liệu lớn.

Trong phạm vi bài giảng, chúng ta quan tâm đến các phương pháp thu thập các nguồn dữ liệu có sẵn từ tương tác của người dùng trong hệ thống mà chúng ta xây dựng hoặc nguồn dữ liệu cho phép tiếp cận trực tiếp từ internet.

- Ghi lịch sử (logging): Là quá trình ghi lại các sự kiện, hành động hoặc thông tin liên quan trong hệ thống, ứng dụng hoặc môi trường máy tính. Việc thu thập dữ liệu dựa trên logging là một phương pháp phổ biến để nắm bắt và lưu trữ thông tin về hoạt động của hệ thống, người dùng. Đây là một công cụ quan trọng cho việc giám sát, phân tích hiệu suất, và hiểu rõ hành vi của người dùng trong các ứng dụng trực tuyến. Chúng ta cần xác định mục tiêu cụ thể của việc thu thập dữ liệu thông qua logging, đặt ra các điểm mốc quan trọng hoặc sự kiện cần được ghi lại. Ví dụ: có thể là giám sát hiệu suất, theo dõi lỗi, hoặc hiểu hành vi người dùng.
- Thu thập dữ liệu từ internet (crawling/scraping): Là quá trình tự động hóa việc trích xuất thông tin từ trang web. Phương pháp này đặc biệt hữu ích khi muốn thu thập dữ liệu từ các trang web công cộng để phân tích, nghiên cứu thị trường, hoặc tạo cơ sở dữ liệu thông tin. Tuy nhiên, cần tuân thủ các quy định và điều kiện của trang web để tránh việc vi phạm quy định về bản quyền và quyền riêng tư. Chúng

ta cần xác định rõ mục tiêu của việc thu thập dữ liệu và xác định loại dữ liệu cần lấy từ trang web, nghiên cứu cấu trúc HTML của trang web để hiểu cách dữ liệu được tổ chức và lưu trữ. Có nhiều công cụ hỗ trợ cho bước thu thập dữ liệu. Chúng ta có thể lựa chọn công cụ phù hợp, ví dụ BeautifulSoup, Scrapy, hoặc Selenium tùy thuộc vào độ phức tạp của trang web và yêu cầu thu thập.

1.4.2 Tiền xử lý dữ liệu

Dữ liệu thực tế ngày nay rất dễ bị ảnh hưởng bởi nhiễu, thiếu sót và không nhất quán do kích thước của chúng thường rất lớn (gigabyte trở lên) và xuất phát từ nhiều nguồn khác nhau, đa dạng. Dữ liệu chất lượng kém sẽ dẫn đến kết quả khai phá dữ liệu kém chất lượng. "Làm thế nào có thể tiền xử lý dữ liệu để cải thiện chất lượng của dữ liệu và cải thiện chất lượng kết quả khai phá dữ liệu? Làm thế nào có thể tiền xử lý dữ liệu để cải thiện hiệu suất và dễ dàng trong quá trình khai phá dữ liệu?".

Có nhiều kỹ thuật trong tiền xử lý dữ liệu. Việc làm sạch dữ liệu (data cleaning) có thể được áp dụng để loại bỏ nhiễu và sửa các không nhất quán trong dữ liệu. Tích hợp dữ liệu (data integration) kết hợp dữ liệu từ nhiều nguồn thành một kho dữ liệu nhất quán như một kho dữ liệu. Giảm dữ liệu (data reduction) có thể giảm kích thước dữ liệu bằng cách tổng hợp, loại bỏ các đặc trưng dư thừa hoặc gom cụm. Biến đổi dữ liệu (data transformation) (ví dụ, chuẩn hóa) có thể được áp dụng, trong đó dữ liệu được chia tỷ lệ để nằm trong một khoảng nhỏ hơn, chẳng hạn từ 0.0 đến 1.0. Điều này có thể cải thiện độ chính xác và hiệu suất của các thuật toán khai phá dữ liệu liên quan đến độ đo khoảng cách. Các kỹ thuật này không phải là độc lập lẻ; chúng có thể hoạt động cùng nhau. Ví dụ, việc làm sạch dữ liệu có thể bao gồm các biến đổi để sửa các dữ liệu sai, chẳng hạn như biến đổi tất cả các mục cho một trường ngày thành một định dạng chung.

Làm sạch dữ liệu

Các quy trình làm sạch dữ liệu gồm các bước để "làm sạch" dữ liệu bằng cách điền vào các giá trị thiếu, làm mịn dữ liệu nhiễu, xác định hoặc loại bỏ các giá trị ngoại lệ và giải quyết sự không nhất quán. Nếu dữ liệu có vấn đề, kết quả của việc khai phá dữ liệu khó có thể tin tưởng. Hơn nữa, dữ liệu không sạch có thể tạo ra sự nhầm lẫn trong quy